

UNIT II

2

Event Handling using AWT / Swing Components

Syllabus

Event Handling : Events, Event sources, Event classes, Event Listeners, Delegation event model, handling mouse and keyboard events, Adapter classes, inner classes. The AWT class hierarchy, user interface components - labels, button, canvas, scrollbars, text components, checkbox, checkbox groups, choices, lists panels - scroll pane, dialogs, menu bar, graphics, layout manager - layout manager types - boarder, grid, flow, card and grib bag.

Contents

- 2.1 Events
- 2.2 Event Sources
- 2.3 Event Classes
- 2.4 Event Listeners
- 2.5 Delegation Event Model
- 2.6 Handling Mouse Events
- 2.7 Handling Keyboard Events
- 2.8 Adapter Classes
- 2.9 Inner Classes
- 2.10 What is Abstract Windowing Toolkit ?
- 2.11 The AWT Class Hierarchy
- 2.12 Limitations of AWT
- 2.13 User Interface Components
- 2.14 Dialogs
- 2.15 Menu bar
- 2.16 Programming Examples based on AWT Components and Event Handling
- 2.17 Graphics
- 2.18 Layout Manager
- 2.19 Multiple Choice Questions

Part I : Event Handling

2.1 Events

Event is a changing state of an object. For example - The user clicks the mouse button or user presses the key on the keyboard.

2.2 Event Sources

- An event source is the object that generates the event. For example if you click a button an **ActionEvent** object is generated.
- The object of the **ActionEvent** class has all corresponding information about this event. Here button is an event source.
- Various event sources can be **button, checkbox, textbox, scrollbars** and so on.

2.3 Event Classes

- Event classes are the classes responsible for handling events in the event handling mechanism.
- The **EventObject** class is at the top of the event class hierarchy. It belongs to the **java.util** package. And other event classes are present in **java.awt.event** package.
- The **getSource()** and **toString()** are the methods of the **EventObject** class.
- There is **getId()** method belonging to **java.awt.event** package returns the **nature of the event**.
- For example, if a keyboard event occurs, you can find out whether the event was key press or key release from the event object.
- Various event classes that are defined in **java.awt.event** class are -
 1. An **ActionEvent** object is generated when a component is activated. For example if a button is pressed or a menu item is selected then this event occurs.
 2. An **AdjustmentEvent** object is generated when scrollbars are used.
 3. A **TextEvent** object is generated when text of a component or a text field is changed.
 4. A **ContainerEvent** object is generated when component are added or removed from container.
 5. A **ComponentEvent** object is generated when a component is resized, moved, hidden or made visible.
 6. An **ItemEvent** is generated when an item from a list is selected. For example a choice is made or if checkbox is selected.

7. A **FocusEvent** object is generated when component receives keyboard focus for input.
8. A **KeyEvent** object is generated when key on keyboard is pressed or released.
9. A **WindowEvent** object is generated when a window activated, maximized or minimized.
10. A **MouseEvent** object is generated when a mouse is clicked, moved, dragged, released.

2.3.1 ActionEvent Class

Description	When an event gets generated due to pressing of button, or by selecting menu item or by selecting an item, this event occurs.
Constructors	ActionEvent(Object source, int id, string command, long when, int modifier) The <i>source</i> indicates the object due to which the event is generated. The <i>id</i> which is used to identify the type of event. The <i>command</i> is a string that specifies the command that is associated with the event. The <i>when</i> denotes the time of event. The <i>modifier</i> indicates the modifier keys such as ALT, CNTRL, SHIFT that are pressed when an event occurs.
Methods	• String getActionCommand() : This method is useful for obtaining the <i>command</i> string which is specified during the generation of event. • int getModifiers() : This method returns the value which indicates the type of key being pressed at the time of event. • long getWhen() : It returns the time at which the event occurs.
Constants	There are four constants that are used to indicate the modifier keys being pressed. These constants are CTRL_MASK, SHIFT_MASK, META_MASK and ALT_MASK.

2.3.2 ItemEvent Class

Description	This event gets caused when an item is selected or deselected.
Constructors	ItemEvent(ItemSelectable, int, Object, int) Constructs a ItemSelectEvent object with the specified ItemSelectable source, type, item and item select state.
Methods	1) getItem() : It returns the item where the event occurred. 2) getItemSelectable() : Returns the ItemSelectable object where this event originated. 3) getStateChange() : Returns the state change type which generated the event.
Constants	1) ITEM_FIRST : Marks the first integer id for the range of item 2) ITEM_LAST : Marks the last integer id for the range of item

- | |
|--|
| 3) ITEM_STATE_CHANGED : The item state changed event type
4) SELECTED : The item selected state change type
5) DESELECTED : The item de-selected state change type |
|--|

2.3.3 KeyEvent Class

Description	This event is generated when key on the keyboard is pressed or released.
Constructors	KeyEvent(Component source, int type, long t, int modifiers, int code) The <i>source</i> is a reference to the component that generates the event. The <i>type</i> specifies the type of event. The <i>t</i> denotes the system time at which the event occurs. The <i>modifiers</i> represent the modifier keys such as ALT, CNTRL, SHIFT that are pressed when an event occurs. The <i>code</i> represents the virtual keycode such as VK_UP, VK_ESCAPE and so on.
Methods	char getKeyChar() : It returns the character when a key is pressed.
Constants	KEY_PRESSED : This event is generated when the key is pressed. KEY_RELEASED : This event is generated when the key is released. KEY_TYPED : This event is generated when the key is typed.

2.3.4 MouseEvent Class

Description	This event occurs when mouse action occurs in a component. It reacts for both mouse event and mouse motion event. - Mouse Events - A mouse button is pressed - A mouse button is released - A mouse button is clicked (pressed and released) - The mouse cursor enters a component - The mouse cursor exits a component - Mouse Motion Events - The mouse is moved - The mouse is dragged
Methods	int getButton() : It returns which of the mouse button has changed the state. int getClickCount() : It returns number of mouse clicks. Point getLocationOnScreen() : Returns the absolute x, y position at that event. Point getPoint() : Returns the x,y position of the event relative to the source component. int getX() : Returns the horizontal x position of the event relative to the source component. int getY() : Returns the vertical y position of the event relative to the source component.

Fields	Following are the fields for java.awt.event.MouseEvent class : • static int BUTTON1 --Indicates mouse button #1; used by getButton() • static int BUTTON2 --Indicates mouse button #2; used by getButton() • static int BUTTON3 --Indicates mouse button #3; used by getButton() • static int MOUSE_CLICKED --The "mouse clicked" event • static int MOUSE_DRAGGED --The "mouse dragged" event • static int MOUSE_ENTERED --The "mouse entered" event • static int MOUSE_EXITED --The "mouse exited" event • static int MOUSE_FIRST --The first number in the range of ids used for mouse events • static int MOUSE_LAST -- The last number in the range of ids used for mouse events • static int MOUSE_MOVED --The "mouse moved" event • static int MOUSE_PRESSED -- The "mouse pressed" event • static int MOUSE_RELEASED --The "mouse released" event • static int MOUSE_WHEEL --The "mouse wheel" event • static int NOBUTTON --Indicates no mouse buttons; used by getButton() static int VK_WINDOWS --Constant for the Microsoft Windows "Windows" key.
---------------	---

2.3.5 TextEvent Class

Description	This event indicates that object's text is changed.
Constructor	TextEvent(Object source, int id) It constructs a TextEvent object.
Methods	String paramString() : Returns a parameter string identifying this text event.
Fields	• TEXT_FIRST : The first number in the range of ids used for text events. • TEXT_LAST : The last number in the range of ids used for text events. • TEXT_VALUE_CHANGED : This event id indicates that object's text changed.

2.3.6 WindowEvent Class

Description	This is an event that indicates that a window has changed its status.
Constructor	WindowEvent(Window source, int id) : Constructs a WindowEvent object. WindowEvent(Window source, int id, int oldState, int newState) : Constructs a WindowEvent object with the specified previous and new window states.
Methods	1) int getNewState() : It returns the new state of the window. 2) int getOldState() : It returns the previous state of the window. 3) Window getOppositeWindow() : Returns the other Window involved in this focus or activation change. 4) Window getWindow() : Returns the originator of the event.

Fields

- **WINDOW_ACTIVATED** : The window-activated event type.
- **WINDOW_CLOSED** : The window closed event.
- **WINDOW_DEACTIVATED** : The window-deactivated event type.
- **WINDOW_FIRST** : The first number in the range of ids used for window events.
- **WINDOW_LAST** : The last number in the range of ids used for window events.
- **WINDOW_OPENED** : The window opened event.
- **WINDOW_STATE_CHANGED** : The window-state-changed event type.

2.4 Event Listeners

- The task of handling an event is carried out by **event listener**.
- When an event occurs, first of all an event object of the appropriate type is created. This object is then passed to a **Listener**.
- A listener must **implement the interface** that has the method for event handling.
- The **java.awt.event** package contains definitions of all event classes and **listener interface**.
- **An Interface** contains constant values and method declaration.
- The methods in an interface are only declared and not implemented, i.e. the methods do not have a body.
- The interfaces are used to define behavior on occurrence of event that can be implemented by any class anywhere in the class hierarchy.

2.4.1 ActionListener Interface

- This interface defines the method **actionPerformed()** which is invoked when an **ActionEvent** occurs.
- The **syntax** of this method is

```
void actionPerformed(ActionEvent act)
```

where **act** is an object of **ActionEvent** class.

2.4.2 ItemListener Interface

- This interface is used when an item from a list is selected. For example a choice is made or if checkbox is selected.
- The **itemStateChanged()** is the only method defined by the **ItemListener** interface.
- The **syntax** is

```
void itemStateChanged(ItemEvent It)
```

2.4.3 KeyListener Interface

- This interface is defining the events such as **keyPressed()**, **keyReleased()** and **keyTyped()** are used.
- These methods are useful for key press, key release and when you type some characters.

```
void keyPressed(KeyEvent k)
void keyReleased(KeyEvent k)
void keyTyped(KeyEvent k)
```

2.4.4 MouseListener Interface

- This interface defines five important methods for various activities such as mouse click, press, released, entered or exited. These are

```
void mouseClicked(MouseEvent m)
void mousePressed(MouseEvent m)
void mouseReleased(MouseEvent m)
void mouseEntered(MouseEvent m)
void mouseExited(MouseEvent m)
```

2.4.5 MouseMotion Interface

- For handling mouse drag and mouse move events the required methods are defined by MouseMotionListener interface. These methods are

```
void mouseDragged(MouseEvent m)
void mouseMoved(MouseEvent m)
```

2.4.6 TextListener Interface

- An event of type TextEvent is generated when a value in textfield or textarea is entered or edited.
- The methods used by TextListener Interface are

```
public String paramString():
public void textValueChanged(TextEvent e):
```

2.4.7 WindowListener Interface

- There are seven methods in which are related to windows activation and deactivation.

```
void windowOpened(WindowEvent w)
void windowClosed(WindowEvent w)
void windowClosing(WindowEvent w)
void windowActivated(WindowEvent w)
void windowDeactivated(WindowEvent w)
```

```
void windowIconified(WindowEvent w)
void windowDeiconified(WindowEvent w)
```

2.5 Delegation Event Model

- **Basic Concept :** Event delegation model is used for understanding the event and for processing it. The event-handler method takes the Event object as a parameter. For handling particular event specific object of event must be mentioned.
- There are four main components based on this model are

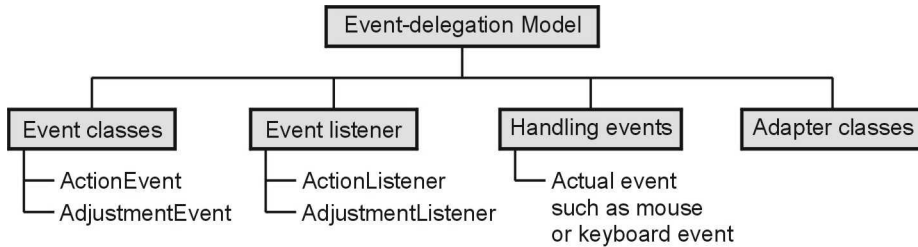


Fig. 2.5.1 Components of event delegation model

Advantages of Event Delegation Model

Following are the advantages of event delegation model -

1. In event delegation model the events are handled using objects. This allows a clear separation between the usage of the components and the design.
 2. It accelerates the performance of the application in which multiple events are used.
- Let us see how an event gets handled with the help of programming example.

Example : Handling Button Click

- The buttons are sometimes called as push buttons. We can associate some event on button click. That means on clicking the button, certain event gets triggered to perform required task. Following example illustrates this idea.

Example 2.5.1 Write a Java program to toggle the background color on every click of button.

Solution :

Java Program[Button1.java]

```
//This program alternatively changes the background color
//After every click of button
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="Button1" width=350 height=200>
```



```
</applet>
*/
public class Button1 extends Applet

implements ActionListener
{

    Button button=new Button("change the color");
    boolean flag=true;
    public void init()
    {
        add(button);
        button.addActionListener(this);
    }
    public void paint(Graphics g)
    {
        if(flag)
            setBackground(Color.yellow);

        else

            setBackground(Color.red);
    }
    public void actionPerformed(ActionEvent e)
    {
        String str=e.getActionCommand();
        if(str.equals("change the color"))
        {
            flag=!flag;
            //toggle the flag values on every click of button
            repaint();
        }
    }
}
```

Event Listner Interface

Placing the button control

Invoking the button click Event

ActionListner Interface defines action performed method. Using this method event is handled

Event gets handled here

Output

D:\>Appletviewer Button1.java



Program Explanation

In above program,

- 1) When a button is clicked the action of changing the background color occurs. Hence this program must implement the **ActionListener** interface.
- 2) This listener class must have a method called **actionPerformed** which has a parameter an object-**ActionEvent**.
- 3) By invoking the method **getActionCommand** we can recognise that the event has occurred by clicking the button.

2.6 Handling Mouse Events

- In order to handle any event, we need to use Listener interfaces. For handling the mouse events the **MouseListener** and **MouseMotionListener** interfaces are used.

Methods of MouseListener interface

- There are 5 abstract methods used under **MouseListener** interface and those are -
 1. public abstract void mouseClicked(MouseEvent e);
 2. public abstract void mouseEntered(MouseEvent e);
 3. public abstract void mouseExited(MouseEvent e);
 4. public abstract void mousePressed(MouseEvent e);
 5. public abstract void mouseReleased(MouseEvent e);

Methods of MouseMotionListener interface

- There are 2 abstract methods used under **MouseMotionListener** interface and those are -
 1. public abstract void mouseDragged(MouseEvent e);
 2. public abstract void mouseMoved(MouseEvent e);

Example Program

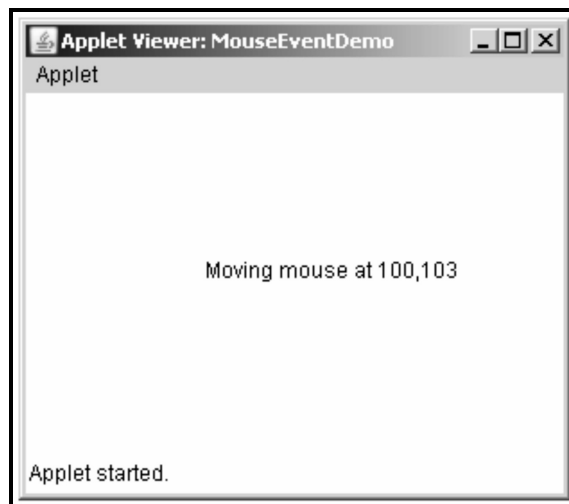
Java Program[MouseEventDemo.java]

```
/*
This is a Java program which is for handing mouse events
*/
import java.awt.*;
import java.applet.*;
import java.awt.event.*;
/*
<applet code="MouseEventDemo" width=300 height=200>
```

```
</applet>
*/
public class MouseEventDemo extends Applet
implements MouseListener, MouseMotionListener
{
    String msg="";
    int xposition=0,yposition=0;
    public void init()
    {
        addMouseListener(this);
        addMouseMotionListener(this);
    }
    public void mouseClicked(MouseEvent m)
    {
        xposition=m.getX();
        yposition=m.getY();
        msg="mouse Clicked";
        repaint();
    }
    public void mousePressed(MouseEvent m)
    {
        xposition=m.getX();
        yposition=m.getY();
        msg="Pressing mouse button";
        repaint();
    }
    public void mouseReleased(MouseEvent m)
    {
        xposition=m.getX();
        yposition=m.getY();
        msg="Releasing mouse button";
        repaint();
    }
    public void mouseEntered(MouseEvent m)
    {
        xposition=0;
        yposition=190;
        msg="mouse Entered";
        repaint();
    }
    public void mouseExited(MouseEvent m)
    {
        xposition=0;
```

```
yposition=190;
msg="mouse Exited";
repaint();
}
public void mouseDragged(MouseEvent m)
{
    xposition=m.getX();
    yposition=m.getY();
    msg="Dragging mouse at "+xposition+","+yposition;
    repaint();
}
public void mouseMoved(MouseEvent m)
{
    xposition=m.getX();
    yposition=m.getY();
    msg="Moving mouse at "+xposition+","+yposition;
    repaint();
}
public void paint(Graphics g)
{
    g.drawString(msg,xposition,yposition);
}
}
```

Output



Program Explanation

In above program, we have used

```
import java.awt.event.*;
```

because various commonly used events are defined in the package **java.awt.event**.

- The applet has to register itself as a listener to various events (*here the same applet acts as a event source as well as event listener*). Hence inside **init()** method applet registers itself as a listener by following statements

```
addMouseListener(this);  
addMouseMotionListener(this);
```

- And then simple methods of mouse events are defined. The **getX()** and **getY()** methods return the current x and y positional values. To each of these methods an object of **MouseEvent** is passed which is shown by a variable 'm'.

Example 2.6.1 *What method is used to distinguish between single, double and triple mouse clicks ? Illustrate.*

Solution : public int getClickCount() : This method returns the number of mouse clicks. Hence we can check -

```
if (mouseEvent.getClickCount() == 1)  
{  
    System.out.println("Single Click");  
}  
else if (mouseEvent.getClickCount() == 2)  
{  
    System.out.println("Double Click");  
}  
else if (mouseEvent.getClickCount() == 3)  
{  
    System.out.println("Triple Click");  
}
```

2.7 Handling Keyboard Events

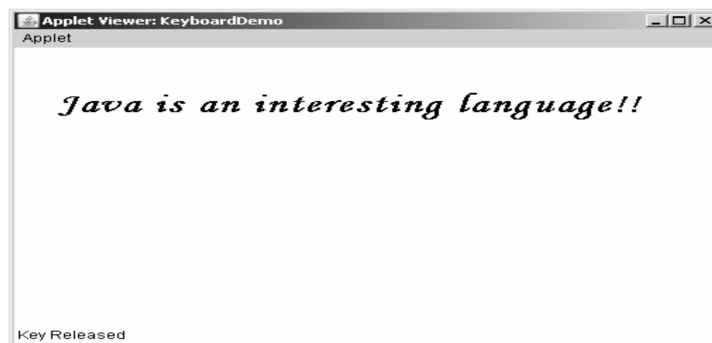
- When key from a keyboard is pressed then it causes an event.
- There are three commonly used methods from **KeyListener** interface and those are
 1. **keyPressed()**,
 2. **keyReleased()** and
 3. **keyTyped()**.
- The use of these methods is shown by following Java program -

Java Program [KeyboardDemo.java]

```
import java.awt.*;  
import java.applet.*;  
import java.awt.event.*;  
/*  
<applet code="KeyboardDemo" width=500 height=300>
```

```
</applet>
*/
public class KeyboardDemo extends Applet
implements KeyListener
{
    String msg="";
    public void init()
    {
        addKeyListener(this);
        requestFocus();
    }
    public void keyPressed(KeyEvent k)
    {
        showStatus("Key Pressed");
    }
    public void keyReleased(KeyEvent k)
    {
        showStatus("Key Released");
    }
    public void keyTyped(KeyEvent k)
    {
        Font f;
        f=new Font("Monotype Corsiva",Font.BOLD,30);
        msg+=k.getKeyChar();
        setFont(f);
        repaint();
    }
    public void paint(Graphics g)
    {
        g.drawString(msg,30,70);
    }
}
```

Output



Program Explanation : In above program, we have used -

- 1) The **keyPressed()** event is for handling the event of pressing a key, similarly **keyReleased()** event is for handling the event of release of key.
- 2) Using **keyTyped()** method we can display the character typed on the screen. The only needed method for this is **getKeyChar()** which returns the currently typed character.
- 3) Last but not the least, all these methods require one important method that has to be invoked in **init()** function and that is **requestFocus()**. This method has to be invoked to gain the focus in the **init** method and whenever you want to implement **KeyListener** interface.

2.8 Adapter Classes

- It is basically a class in Java that implements an interface with a set of dummy methods.
- The famous adapter classes in Java API are **WindowAdapter**, **ComponentAdapter**, **ContainerAdapter**, **FocusAdapter**, **KeyAdapter**, **MouseAdapter** and **MouseMotionAdapter**.
- Whenever your class implements such interface, you have to implements all of the seven methods.
- **WindowAdapter** class implements **WindowListener** interface and make seven empty implementation.
- When you class subclass **WindowAdapter** class, you may choose the method you want without restrictions.
- The following give such an example.

```
public interface WindowListener {
    public void windowClosed(WindowEvent e);
    public void windowOpened(WindowEvent e);
    public void windowIconified(WindowEvent e);
    public void windowDeiconified(WindowEvent e);
    public void windowActivated(WindowEvent e);
    public void windowDeactivated(WindowEvent e);
    public void windowClosing(WindowEvent e);
}

public class WindowAdapter implements WindowListener{
    public void windowClosed(WindowEvent e){}
    public void windowOpened(WindowEvent e){}
    public void windowIconified(WindowEvent e){}
    public void windowDeiconified(WindowEvent e){}
```

```
public void windowActivated(WindowEvent e){}  
public void windowDeactivated(WindowEvent e){}  
public void windowClosing(WindowEvent e){}  
}
```

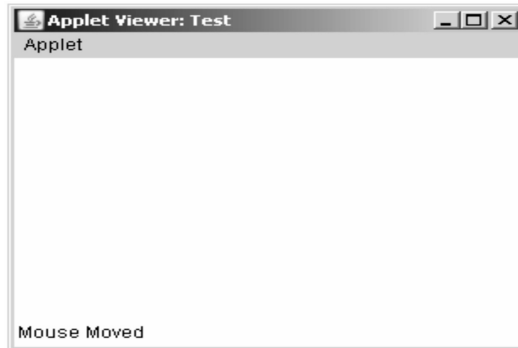
- You can add adapter class as subclass and override just the methods you need.
- Here is a simple Java program for handling mouse events.

```
import java.awt.*;  
import java.awt.event.*;  
import java.applet.*;  
/*  
<applet code="Test" width=300 height=200>  
</applet>  
*/  
public class Test extends Applet  
{  
    public void init()  
    {  
        //defining the adapter classes  
        //adapter1 is for MouseListener  
        //adapter2 is for MouseMotionListener  
  
        addMouseListener(new adapter1(this));  
        addMouseMotionListener(new adapter2(this));  
    }  
}  
class adapter1 extends MouseAdapter  
{  
    //object of main Test class  
    Test obj;  
    public adapter1(Test obj)  
    {  
        this.obj=obj;  
    }  
    //method belonging to MouseListener interface  
    public void mouseClicked(MouseEvent m)  
    {  
        obj.showStatus("Mouse clicked");  
    }  
}  
class adapter2 extends MouseMotionAdapter  
{  
    Test obj;  
    public adapter2(Test obj)  
    {
```



```
this.obj=obj;
}
//method belonging to MouseMotionListener interface
public void mouseMoved(MouseEvent m)
{
    obj.showStatus("Mouse Moved");
}
}
```

Output



Program Explanation :

- 1) In this program we have used two interfaces **MouseListener** and **MouseMotionListener**.
- 2) Two adapter classes are created. The **adapter1** class is for **MouseListener** interface and the **adapter2** class is for **MouseMotionListener** interface.
- 3) Note that, in the class **Test** we have written **init** method in which two adapter classes are created and registered as Listener using following statements

```
addMouseListener(new adapter1(this));
addMouseMotionListener(new adapter2(this));
```

- 4) Then we have defined **adapter1** class in which object for adapter class **adapter1** is initialised. Note that we have written **mouseClicked** method in **adapter1** class because **mouseClicked** method is belonging to the **MouseListener** interface and the adapter class **adapter1** is for **MouseListener**.
- 5) Same is true for the **adapter2** class which is an adapter class for **MouseMotionListener**. Note that in the definition of **adapter2** class **mouseMoved** method is written because **mouseMoved** method is belonging to the **MouseMotionListener**.
- 6) On running the above code, appropriate status will be shown on applet on encountering particular mouse event (either mouse click or mouse move).

Review Question

1. What are the various adapter classes that implements commonly used Listener interfaces ?
Write a sample Java program to demonstrate an Adapter.

2.9 Inner Classes

Definition : Inner classes are the nested classes. That means these are the classes that are defined inside the other classes.

The syntax of defining the inner class is -

```
Access_modifier class OuterClass
{
    //code
    Access_modifier class InnerClass
    {
        //code
    }
}
```

Properties of inner classes

Following are some properties of inner class -

- The outer class can inherit as many number of inner class objects as it wants.
- If the outer class and the corresponding inner class both are **public** then any other class can create an instance of this inner class.
- The inner class objects do not get instantiated with an outer class object.
- The outer class can call the private methods of inner class.
- Inner class code has free access to all elements of the outer class object that contains it.
- If the inner class has a variable with same name then the outer class's variable can be accessed like this -

```
outerclassname.this.variable_name
```

There are **four types of inner classes** -

2.9.1 Static Member Classes

- This inner class is defined as the static member variable of another class.
- Static members of the outer class are visible to the **static inner** class.
- The non-static members of the outer class are not available to inner class.

Syntax

```
Access_modifier class OuterClass
{
    //code
    public static class InnerClass
    {
        //code
    }
}
```

2.9.2 Member Classes

This type of inner class is non-static member of outer class.

Syntax

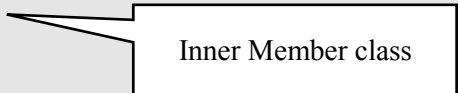
```
Access_modifier class outerclass
{
    //code
    Access_modifier class interclass
    {
        //code
    }
}
```

Example

```
class A
{
    private String name="Ankita";

    class B
    {
        void display()
        {
            System.out.println("Name is: "+name);
        }
    }

    public static void main(String args[])
    {
        A obj=new A();
        A.B in_obj=obj.new B();
        in_obj.display();
    }
}
```



A callout box with the text "Inner Member class" has two lines pointing to the "class B" declaration and its opening curly brace in the code example.

Output

Name is : Ankita

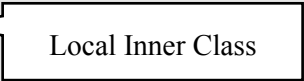
2.9.3 Local Classes

- This class is defined within a Java code just like a local variable.
- Local classes are never declared with an access specifier.
- The scope inner classes is always restricted to the block in which they are declared.
- The local classes are completely hidden from the outside world.

Syntax

```
Access_modifier class OuterClass
{
    //code
    Access_modifier return_type methodname(arguments)
    {
        class InnerClass
        {
            //code
        }
        //code
    }
}
```

Example

```
public class A
{
    private String name="Ankita";//instance variable
    void display()
    {
        class Local 
        {
            void show()
            {
                System.out.println("From Local class: "+name);
            }
        }
        Local obj=new Local();
        obj.show();
    }
    public static void main(String args[])
    {
        A a=new A();
        a.display();
    }
}
```

Output

From Local class: Ankita

2.9.4 Anonymous Classes

- Anonymous class is a local class without any name.
- Anonymous class is a one-shot class- created exactly where needed.
- The anonymous class is created in following situations -
 - When the class has very short body.
 - Only one instance of the class is needed.
 - Class is used immediately after defining it.
- The anonymous inner class can extend the class, it can implement the interface or it can be declared in method argument.

Don't

Programming Example

```
class MyInnerClass implements Runnable
{
    public void run()
    {
        System.out.println("Hello");
    }
}
class DemoClass
{
    public static void main(String[] arg)
    {
        MyInnerClass my=new MyInnerClass();
        Thread th=new Thread(my);
        my.start();
    }
}
```

Review Question

1. What is inner classes ? Explain its types.

Part II : AWT

2.10 What is Abstract Windowing Toolkit ?

- The AWT stands for Abstract Window Toolkit.
- The AWT package contains large number of classes which help to include various **graphical components** in the Java Program.
- The graphical components include text box, buttons, labels, radio buttons and so on.

2.11 The AWT Class Hierarchy

- The AWT classes are arranged in hierarchical manner which is known as AWT Hierarchy. Refer Fig. 2.11.1.

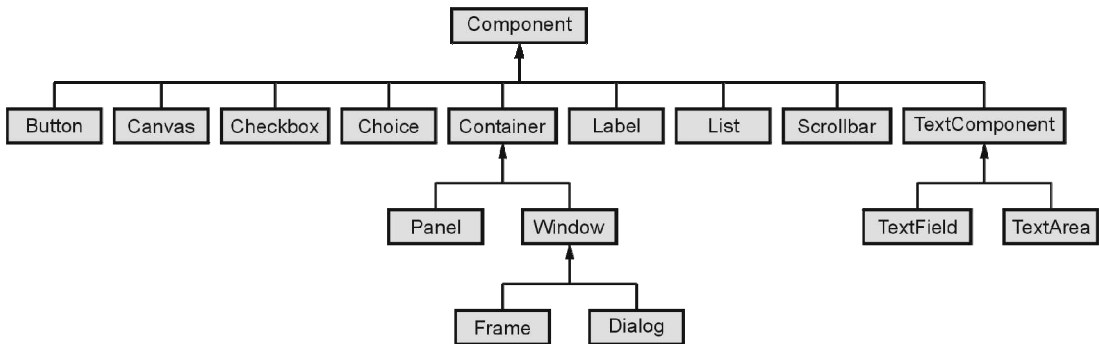


Fig. 2.11.1 AWT hierarchy

- The hierarchy components classes are -
 - **Component** : This is the super class of all the graphical classes from which variety of graphical classes can be derived. It helps in displaying the graphical object on the screen. It handles the mouse and keyboard events.
 - **Container** : This is a graphical component derived from **the component** class. It is responsible for managing the layout and placement of graphical components in the container.
 - **Window** : The top level window without border and without the menu bar is created using the window class. It decides the layout of the window.
 - **Panel** : The panel class is derived from the **container** class. It is just similar to window - without any border and without any menu bar, title bar.
 - **Frame** : This is a top-level window with a border and menu bar. It supports the common window events such as window open, close, activate and deactivate.

Review Question

1. Explain AWT hierarchy in detail

2.12 Limitations of AWT

Following are some limitations of AWT

1. The AWT components are heavyweight in nature.
2. It does not support some advanced components such as trees, tables and so on.
3. It is platform dependable.
4. The buttons of AWT does not support pictures.

2.13 User Interface Components

- There are various graphical components that can be placed on the frame. These components have the classes. These classes have the corresponding methods.
- When we place the components on the frame we need to set the layout of the frame.
- The commonly used layout is **FlowLayout**. The **FlowLayout** means the components in the frame will be placed from left to right in the same manner as they get added.
- Various components that can be placed for designing user interface are -
 1. Label
 2. Buttons
 3. Canvas
 4. Scroll bars
 5. Text Components
 6. Checkbox
 7. Choices
 8. Lists Panels
 9. Dialogs
 10. Menubar
- Let us discuss these components one by one, but before that let us understand how to create a frame on which we can place the components.

2.13.1 Labels

- The syntax of this control is

```
Label (String s)
```

```
Label(String s, int style)
```

- where the **s** of String type represent the string contained by the label similarly in the other label function style is a constant used for the style of label. It can be Label.LEFT, Label.RIGHT and Label.CENTER. Here is a JAVA program which makes use Label.

Example 2.13.1 Write a simple Java program to demonstrate the use of label components.

Solution :

Java Program[Use_Label.java]

```
import java.awt.*;
class Use_Label
{
    public static void main(String[] args)
    {
        Frame fr=new Frame("This Program is for Displaying the Label");
        fr.setSize(400,200);
        fr.setLayout(new FlowLayout());
        fr.setVisible(true);
        Label L1=new Label("OK");
        Label L2=new Label("CANCEL");
        fr.add(L1);
        fr.add(L2);
    }
}
```

```
}  
}
```

Output

```
C:\>javac Use_label.java
```

```
C:\>java Use_label
```



2.13.2 Buttons

- Buttons are sometimes called as **push buttons**. This component contains a label and when it is pressed it generates an event.
- The syntax of this control is

```
Button (String s)
```

Java Program

```
import java.awt.*;  
class Use_Button  
{  
    public static void main(String[] args)  
    {  
        Frame fr=new Frame("This Program is for Displaying the Button");  
        fr.setSize(400,200);  
        fr.setLayout(new FlowLayout());  
        fr.setVisible(true);  
        Button B1=new Button("OK");  
        Button B2=new Button("CANCEL");  
        fr.add(B1);  
        fr.add(B2);  
    }  
}
```

Output



- We can create an array of buttons. Following is a simple Java program which illustrates this ideas -

Java Program[Use_Button_Arr.java]

```
import java.awt.*;
class Use_Button_Arr
{
    public static void main(String[] args)
    {
        int i;
        Frame fr=new Frame("This Program is for Displaying the Buttons");
        fr.setSize(400,200);
        fr.setLayout(new FlowLayout());
        fr.setVisible(true);
        Button buttons[]=new Button[5];
        String Fruits[]={"Mango","Orange","Banana","Apple","Strawberry"};
        for(i=0;i<5;i++)
        {
            buttons[i]=new Button(""+Fruits[i]);
            fr.add(buttons[i]);
        }
    }
}
```

Output



2.13.3 Canvas

- Canvas is a **special area** created on the frame.
- The canvas is specially used for drawing the graphical components such as oval, rectangle, line and so on.

- Various methods of this class are

Method	Description
void setSize(int width, int height)	This method sets the size of the canvas for given width and height.
void setBackground(Color c)	This method sets the background color of the canvas.
void setForeground(Color c)	This method sets the color of the text.

- Following is a simple Java program which shows the use of canvas -

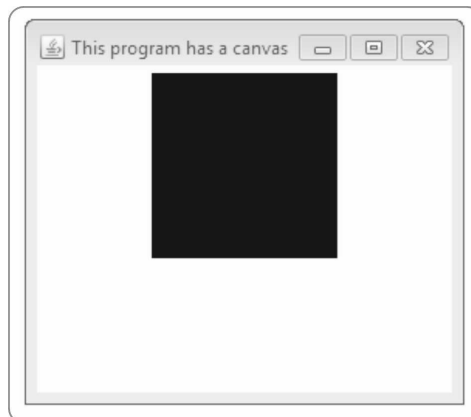
Java Program[Use_Canvas.java]

```
import java.awt.*;
class Use_Canvas
{
    public static void main(String[] args)
    {
        Frame Fr = new Frame("This program has a canvas");
        Canvas C1 = new Canvas();
        C1.setSize(120,120);
        C1.setBackground(Color.blue);
        Fr.setLayout(new FlowLayout());
        Fr.setSize(250,250);
        Fr.setVisible(true);
        Fr.add(C1);
    }
}
```

Output

```
C:\>javac Use_Canvas.java
```

```
C:\>java Use_Canvas
```



2.13.4 Scrollbars

- Scrollbar can be represented by the slider widgets.
- There are two styles of scroll bars - Horizontal scroll bar and vertical scroll bar.
- Following program shows the use of this component.

Java Program

```
import java.awt.*;
class Use_ScrollBars
{
    public static void main(String[] args)
    {

        Frame Fr = new Frame("This program has a scrollbars");
        Scrollbar HSelector = new Scrollbar(Scrollbar.HORIZONTAL);
        Scrollbar VSelector = new Scrollbar(Scrollbar.VERTICAL);

        Fr.setLayout(new FlowLayout());
        Fr.setSize(300,300);
        Fr.setVisible(true);
        Fr.add(HSelector);
        Fr.add(VSelector);
    }
}
```

Output



2.13.5 Text Components

- In Java there are two controls used for text box - One is **TextField** and the other one is **TextArea**.
- The **TextField** is a slot in which one line text can be entered. In the **TextField** we can enter the string, modify it, copy, cut or paste it. The syntax for the text field is

```
int TextField(int n)
```

where n is total number of characters in the string.

- The **TextArea** control is used to handle multi-line text. The syntax is -

```
TextArea(int n,int m)
```

where n is for number of lines and m is for number of characters.

Following is a Java program which illustrates the use of **TextField** and **TextArea**.

Java Program[Use_TxtFld.java]

```
import java.awt.*;
class Use_TxtFld
{
    public static void main(String[] args)
    {
        int i;
        Frame fr=new Frame("This Program is for Displaying the TextField");
        fr.setSize(350,300);
        fr.setLayout(new FlowLayout());
        fr.setVisible(true);
        Label L1=new Label("Enter your name here");
        TextField input1=new TextField(10);
        Label L2=new Label("Enter your address here");
        TextArea input2=new TextArea(10,20);
        fr.add(L1);
        fr.add(input1);
        fr.add(L2);
        fr.add(input2);
    }
}
```

Output**2.13.6 Checkbox**

- Checkbox is basically a small box which can be ticked or not ticked.
- In Java we can select particular item using checkbox control.
- This control appears as small box along with label. The label tells us the name of the item to be selected.
- The syntax of checkbox is as given below -

`Checkbox(String label)`

where *label* denotes the label associated with each checkbox.

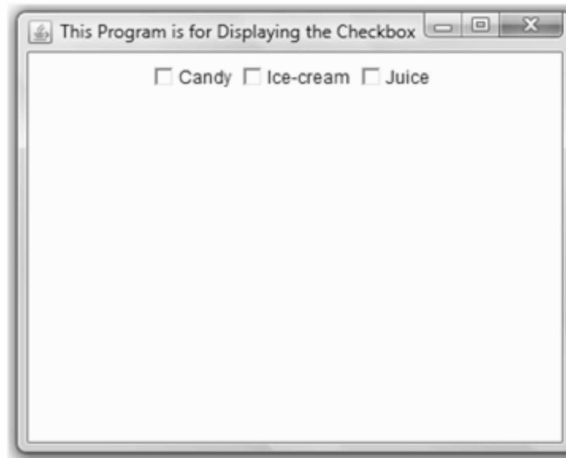
- To get the state of the checkbox the `getState()` method can be used.

Java Program[Use_ChkBox.java]

```
import java.awt.*;
class Use_ChkBox
{
    public static void main(String[] args)
    {
        int i;
        Frame fr=new Frame("This Program is for Displaying the Checkbox");
        fr.setSize(350,300);
        fr.setLayout(new FlowLayout());
        fr.setVisible(true);
        Checkbox box1=new Checkbox("Candy");
```

```
Checkbox box2=new Checkbox("Ice-cream");  
Checkbox box3=new Checkbox("Juice");  
fr.add(box1);  
fr.add(box2);  
fr.add(box3);  
}  
}
```

Output



2.13.7 Checkbox Group

- The Checkbox Group component allows the user to make one and only one selection at a time.
- These checkbox groups is also called as **radio buttons**. The syntax for using checkbox groups is -

```
Checkbox(String str ,CheckboxGroup cbg , Boolean val);
```

- Following is a simple Java program which makes use of this control.

Java Program[Use_CheckBoxGr.java]

```
import java.awt.*;  
class Use_CheckBoxGr  
{  
    public static void main(String[] args)  
    {  
  
        Frame Fr = new Frame("This program uses checkbox groups");  
        Fr.setLayout(new FlowLayout());  
        Fr.setSize(300,300);  
        Fr.setVisible(true);  
    }  
}
```

```
CheckboxGroup cbg=new CheckboxGroup();
Checkbox box1=new Checkbox("Candy",cbg,true);
Checkbox box2=new Checkbox("Ice-cream",cbg,false);
Checkbox box3=new Checkbox("Juice",cbg,false);
Fr.add(box1);
Fr.add(box2);
Fr.add(box3);
}
}
```

Output



2.13.8 Choices

- This is a simple control which allows the popup list for selection.
- We have to create an object of type **choice** as follows -

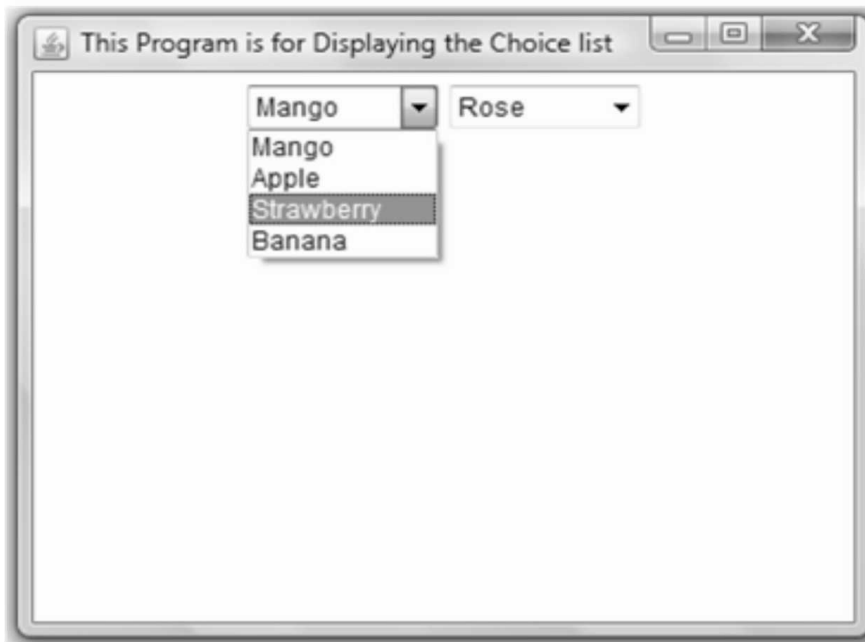
```
Choice obj=new Choice();
```

Java Program[Use_Choice.java]

```
import java.awt.*;
class Use_Choice
{
    public static void main(String[] args)
    {
        int i;
        Frame fr=new Frame("This Program is for Displaying the Choice list");
        fr.setSize(350,300);
        fr.setLayout(new FlowLayout());
        fr.setVisible(true);
        Choice c1=new Choice();
        Choice c2=new Choice();
        c1.add("Mango");
```

```
c1.add("Apple");  
c1.add("Strawberry");  
c1.add("Banana");  
  
c2.add("Rose");  
c2.add("Lily");  
c2.add("Lotus");  
fr.add(c1);  
fr.add(c2);  
  
}  
}
```

Output



2.13.9 List Panels

- **List** is a collection of many items.
- By double clicking the desired item we can select it. Following Java program makes use of this control -

Java Program[Use_List.java]

```
import java.awt.*;  
class Use_List  
{
```



```
public static void main(String[] args)
{
    int i;
    Frame fr=new Frame("This Program is for Displaying the List");
    fr.setSize(350,300);
    fr.setLayout(new FlowLayout());
    fr.setVisible(true);
    List flower=new List(4,false);
    flower.add("Rose");
    flower.add("Jasmine");
    flower.add("Lotus");
    flower.add("Lily");
    fr.add(flower);
}
```

Output



2.14 Dialogs

- Dialog control represents a top-level window with a title and a border used to take some form of input from the user.
- The dialog boxes does not have maximize and minimize buttons.

Constructor for using Dialog Box

Constructor	Description
Dialog(Dialog owner)	Creates modeless dialog window with a frame owner.
Dialog(Dialog owner, String title)	Creates modeless dialog window with a frame owner and string title.
Dialog(Dialog owner, String title, boolean modal)	Creates modeless dialog window with a frame owner, string title and modality to be true or false i.e. if the dialogbox is modal or modeless

What is modal and modeless Dialog Window ?**Modal Dialog Box**

- A Modal dialog box is one that the user must first close in order to have access to any other framed window or dialog box of the same application.

Modeless Dialog Box

- A dialog box is referred to as modeless if the user does not have to close it in order to continue using the application that owns the dialog box.

Java Program[DialogBoxProg.java]

```
import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.*.*;
class DialogBoxProg extends Frame
{
    public static void main(String[] args)
    {
        Dialog d;
        Frame frame = new Frame();
        d=new Dialog(frame,"Dialog Box Demo",true);
        d.add( new Label ("This is a simple dialog box."));
        d.setSize(300,300);
        d.setVisible(true);
        frame.setSize(330,250);
        frame.setVisible(true);
    }
}
```

output**2.14.1 File Dialog**

- The `FileDialog` class displays a dialog window from which the user can select a file.
- Since it is a modal dialog, when the application calls its `show` method to display the dialog, it blocks the rest of the application until the user has chosen a file.
- **Signature for File Dialogbox is**

```
public class FileDialog extends Dialog
```

Methods used by FileDialog

Name	Purpose
<code>getDirectory()</code>	gets the directory of file dialog
<code>getFile()</code>	gets selected file of file dialog
<code>getMode()</code>	indicates whether file dialog box is for loading from a file or saving to a file.
<code>setDirectory(String)</code>	Sets the directory of this file dialog window to be the specified directory.
<code>setFile(String)</code>	Sets the selected file for this file dialog window to be the specified file.
<code>setMode(int)</code>	Sets the mode of the file dialog

Variables used by FileDialog

Name	Purpose
LOAD	This constant value indicates that the purpose of the file dialog window is to locate a file from which to read.
SAVE	This constant value indicates that the purpose of the file dialog window is to locate a file to which to write.

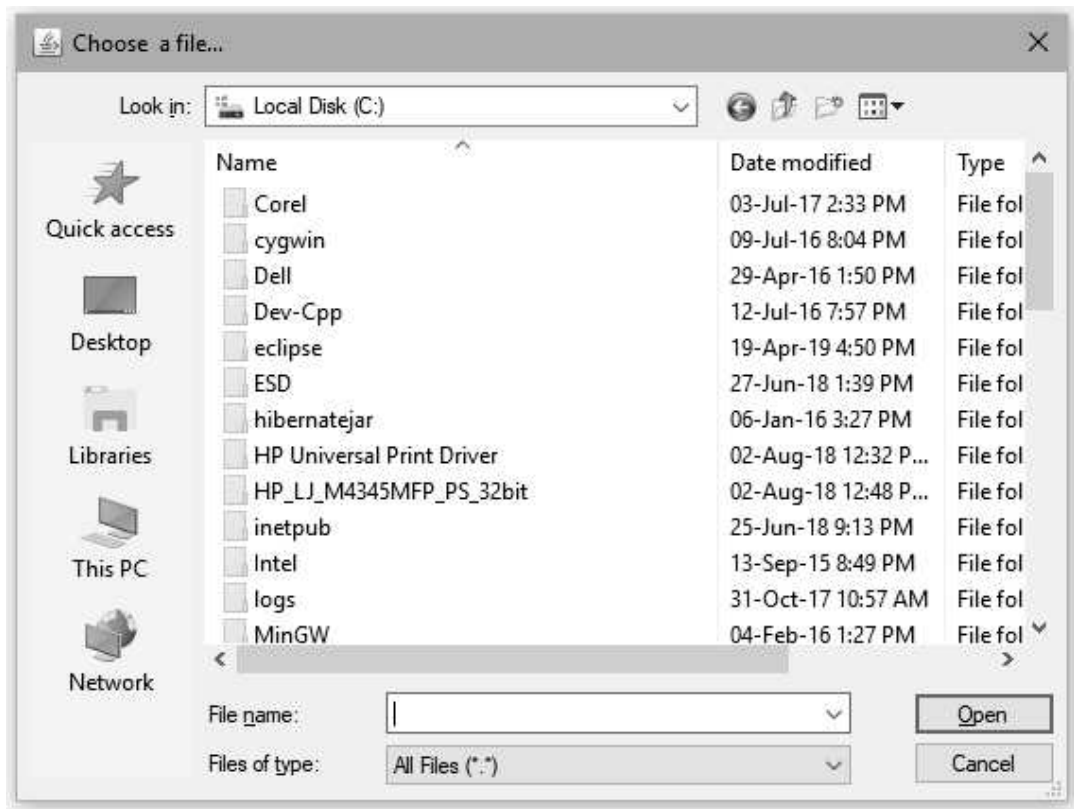
Syntax

```
public FileDialog(Frame parent, String title, int mode)
```

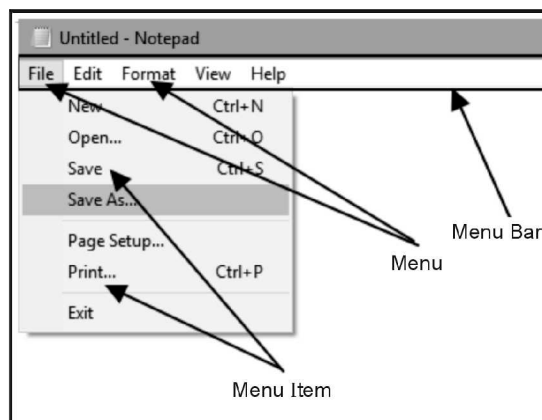
- Creates a file dialog window with the specified title for loading or saving a file.
- If the value of mode is LOAD, then the file dialog is finding a file to read. If the value of mode is SAVE, the file dialog is finding a place to write a file.

Java Program[FileDialog.java]

```
import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.*.*;
class FileDialogBoxProg extends Frame
{
    public static void main(String[] args)
    {
        FileDialog fd;
        Frame frame = new Frame();
        fd=new FileDialog(frame,"Choose a file...",FileDialog.LOAD);
        fd.setDirectory("C:\\");
        fd.setSize(300,300);
        fd.setVisible(true);
        frame.setSize(330,250);
        frame.setVisible(true);
    }
}
```

output**2.15 Menu bar**

- Menus are essential components of any Window based GUI. It allows the user to choose one of several options.
- Menus are created with the help of Menu items and these menus are placed on menubar. Following figure represents Menu, Menubar and Menuitems.



- **Constructor**

For creating Menu, menu items and Menu bar using AWT we use three constructors

Constructor	Description
public MenuBar()	It helps in creating the menubar on which the menus can be added.
public Menu(String title)	The menu items can be created using some title.
public MenuItem(String title)	The menu items are created with suitable title for particular menu.

- **Steps for creating menu**

Creation of menus involves many steps to be followed in an order. Following are the steps.

Step 1 : Create menu bar

Step 2 : Create menus

Step 3 : Create menu items

Step 4 : Add menu items to menus

Step 5 : Add menus to menu bar

Step 6 : Add menu bar to the frame

Following program demonstrates these steps in a simple Java program

Java Program[MenuDemo.java]

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class MenuDemo extends Frame
{
    public static void main(String[] args)
    {
        MenuBar menuBar;
        Menu menu1;
        MenuItem mItem1, mItem2, mItem3;
        Frame frame = new Frame("MenuBar and Menu Demo");

        //Creating a menu bar
        menuBar= new MenuBar();

        //Creating menu
        menu1 = new Menu("File");

        //creating menu items
```

```
mItem1 = new MenuItem("New");
mItem2 = new MenuItem("Open");
mItem3 = new MenuItem("Save");

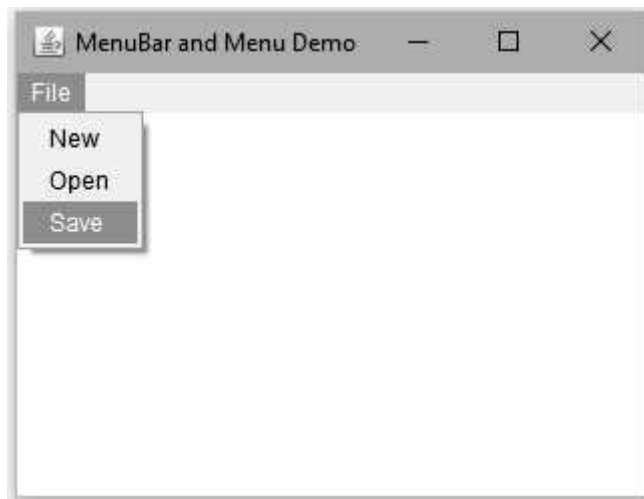
//Adding menu items to the menu
menu1.add(mItem1);
menu1.add(mItem2);
menu1.add(mItem3);

//Adding our menu to the menu bar
menuBar.add(menu1);

//Adding my menu bar to the frame by calling setMenuBar() method
frame.setMenuBar(menuBar);

frame.setSize(330,250);
frame.setVisible(true);
}
```

Output



Program Explanation : In above program,

- 1) We have created a menubar and a menu **File** is added to this menu bar.
- 2) The menu items **New, Open, Save** are added to the **File** menu.
- 3) This set of Menubar, Menu and Menu Item is then added on the frame.

How to add submenus ?

We can add submenu, to a menu. It is illustrated by following program.

Java Program[SubMenuDemo.java]

```
import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.*.*;
class SubMenuDemo extends Frame
{
    public static void main(String[] args)
    {
        MenuBar menuBar;
        Menu menu1,menu2;
        MenuItem mItem1, mItem2, mItem3,mItem4,mItem5;
        Frame frame = new Frame("MenuBar and Menu Demo");

        //Creating a menu bar
        menuBar= new MenuBar();

        //Creating menu
        menu1 = new Menu("File");

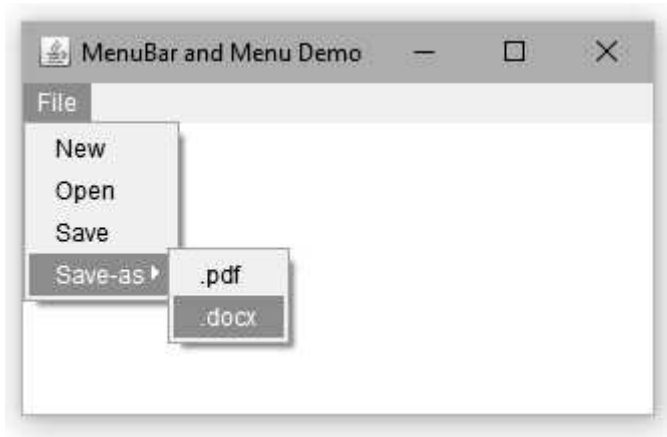
        //creating menu items
        mItem1 = new MenuItem("New");
        mItem2 = new MenuItem("Open");
        mItem3 = new MenuItem("Save");
        //Adding menu items to the menu
        menu1.add(mItem1);
        menu1.add(mItem2);
        menu1.add(mItem3);

        //creating sub menu
        menu2 = new Menu("Save-as");
        //Adding menu items to the submenu
        mItem4 = new MenuItem(".pdf");
        mItem5 = new MenuItem(".docx");
        //Adding menu items to the submenu
        menu2.add(mItem4);
        menu2.add(mItem5);
        //adding submenu to menu
        menu1.add(menu2);
        //Adding menu to the menu bar
        menuBar.add(menu1);
    }
}
```



```
//Adding my menu bar to the frame by calling setMenuBar() method
frame.setMenuBar(menuBar);
frame.setSize(330,250);
frame.setVisible(true);
}
}
```

Output



Review Question

1. Write a program in Java AWT to create Menu and Menu items.

2.16 Programming Examples based on AWT Components and Event Handling

Example 2.16.1 Develop an applet that receives two numerical values as input from the user and then displays the sum of these numbers on the screen. Write the HTML code that calls the applet.

Solution :

Step 1 :

test.html

```
<html>
<body>
<applet code="NumOperations" width=300 height=300>
</applet>
</body>
</html>
```

Step 2 :**NumOperations.java**

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;

public class NumOperations extends Applet implements ActionListener
{
    Label L1,L2,L3;
    TextField T1,T2,T3;
    Button B1;
    public void init()
    {
        L1=new Label("Enter Num1 :");
        add(L1);
        T1=new TextField(15);           //TextField for Num1
        add(T1);

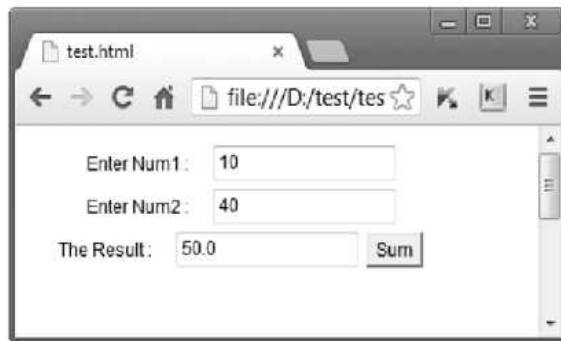
        L2=new Label("Enter Num2 :");
        add(L2);
        T2=new TextField(15);           //TextField for Num2
        add(T2);

        L3=new Label("The Result :");
        add(L3);
        T3=new TextField(15);           //TextField for result
        add(T3);

        B1=new Button("Sum");
        add(B1);                       //Button to invoke addition operation
        B1.addActionListener(this);
    }
    public void actionPerformed(ActionEvent e)
    {
        if(e.getSource()==B1)
        {
            int a=Integer.parseInt(T1.getText());
            int b=Integer.parseInt(T2.getText());
            Float c=Float.valueOf(a+b);
            T3.setText(String.valueOf(c));
        }
    }
}
```

Output

Open **test.html** file on web browser.



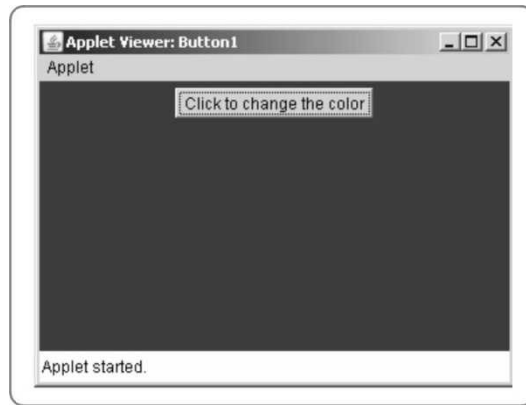
Example 2.16.2 *Write applet program to that alternatively changes the background color after every click of button.*

Solution :

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="Button1" width=350 height=200>
</applet>
*/
public class Button1 extends Applet
{
    Button button=new Button("Click to change the color");
    boolean flag=true;
    public void init()
    {
        add(button);
    }
    public void paint(Graphics g)
    {
        if(flag)
            setBackground(Color.yellow);
        else
            setBackground(Color.red);
    }
    public boolean action(Event e,Object o)
    {
        if(e.target==button)
        {
```

```
flag=!flag;  
//toggle the flag values on every click of button  
repaint();  
return true;  
}  
return false;  
}  
}
```

Output



Program Explanation

In above program,

- First of all we have created a button object 'button' using class **Button**. The string "Click to change the color" is written on the button.

```
Button button=new Button("Click to change the color");
```

- Then in the **init** method we have added button using **add()**.
- Then there is a special method called **action()** which is used to deal with the button clicks. There is no specific call to this method rather it is called automatically when you press a button.
- There are two parameters that are passed to the **action** method, first parameter is an object e of type **Event** and other one is an object of type **Object**.
- The property **target** of the **Event** e is compared with the **button**.

```
if(e.target==button)
```

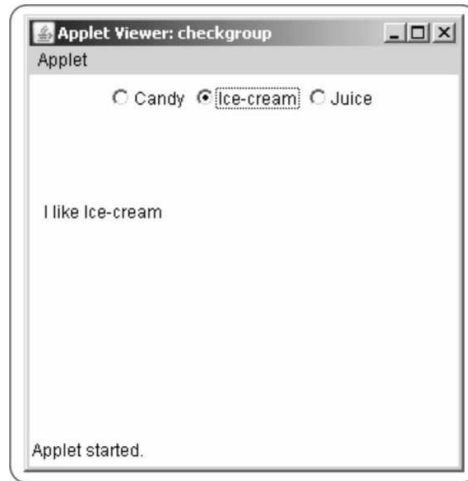
- This helps in finding whether the button is pressed or not.
- We have maintained one variable **flag** which is complemented on each button click. However simply toggling the value of the variable is not sufficient to change the color of the screen, that is why we have called the paint method repeatedly using **repaint()** method.

- In **paint()** method red and yellow background colors are set.

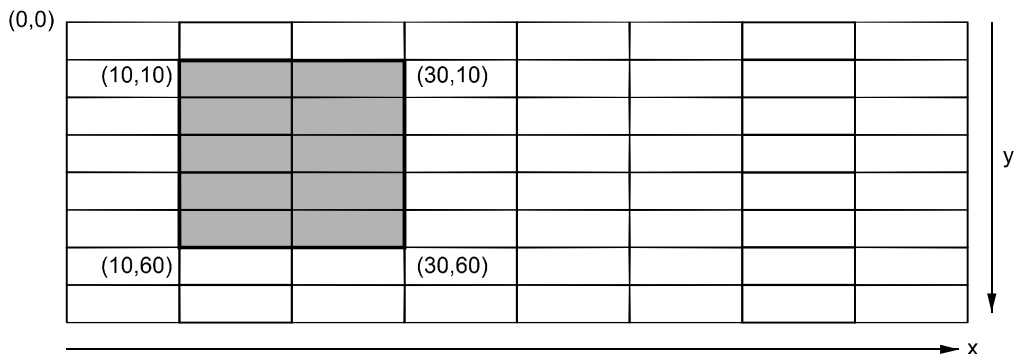
Example 2.16.3 Write a Java program to create AWT radio buttons using check box group.
Explain various event listener interface.

Solution :

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="checkgroup" width=300 height=250>
</applet>
*/
public class checkgroup extends Applet implements ItemListener
{
    String msg=" ";
    CheckboxGroup gr=new CheckboxGroup();
    Checkbox box1=new Checkbox("Candy",gr,true);
    Checkbox box2=new Checkbox("Ice-cream",gr,false);
    Checkbox box3=new Checkbox("Juice",gr,false);
    public void init()
    {
        =
        add(box1);
        add(box2);
        add(box3);
        box1.addItemListener(this);
        box2.addItemListener(this);
        box3.addItemListener(this);
    }
    public void itemStateChanged(ItemEvent e)
    {
        repaint();
    }
    public void paint(Graphics g)
    {
        msg="I like ";
        msg +=gr.getSelectedCheckbox().getLabel();
        g.drawString(msg,10,100);
    }
}
```

Output**2.17 Graphics**

- Java has an ability to draw various graphical shapes. The applet can be used to draw these shapes.
- These objects can be colored using various colors.
- Every applet has its own area which is called **canvas** on which the display can be created.
- The co-ordinate system of Java can be represented by following Fig. 2.17.1.

**Fig. 2.17.1 Co-ordinate system**

- Before drawing the 2D shapes we will need a special area on the frame. This area is called **canvas**. Various methods of this class are -

void setSize(int width,int height) - Sets the size of the canvas of given width and height.

void setBackground(Color c) - Sets the background color of the canvas.

void setForeground(Color c) - Sets the color of the text.

2.17.1 Lines

- Drawing the line is the simplest thing in Java. The syntax is,

```
void drawLine(int x1,int y1,int x2,int y2);
```

Where x1 and y1 represents the starting point of the line and x2 and y2 represents the ending point of the line.

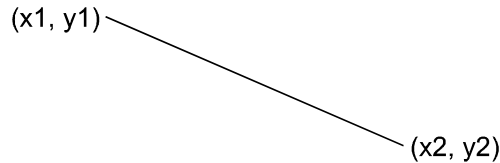
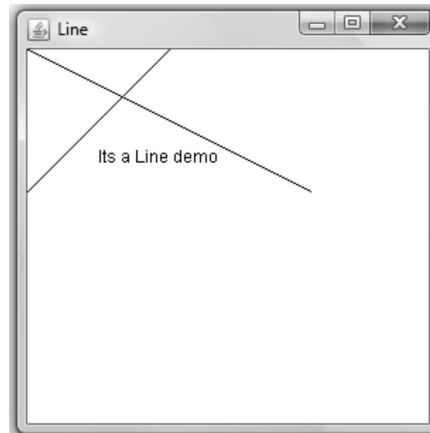


Fig. 2.17.2 Line

Here is a demonstration -

Java Program[LineDemo.java]

```
import java.awt.*;
class LineDemo extends Canvas
{
    public LineDemo()
    {
        setSize(200,200);
        setBackground(Color.white);
    }
    public static void main(String[] args)
    {
        LineDemo obj=new LineDemo();
        Frame fr=new Frame("Line");
        fr.setSize(300,300);
        fr.add(obj);
        fr.setVisible(true);
    }
    public void paint(Graphics g)
    {
        g.drawLine(0,0,200,100);//diagonal line from top-left
        g.drawLine(0,100,100,0);//another diagonal line
        g.drawString("Its a Line demo",50,80);
    }
}
```

Output**2.17.2 Rectangle**

In Java we can draw **two types of rectangles** : normal rectangle and the rectangle with round corners. The syntax of these methods are -

```
void drawRect(int top,int left,int width,int height)
```

```
void drawRoundRect(int top,int left,int width,int height,int xdimeter,int ydimeter)
```

For example

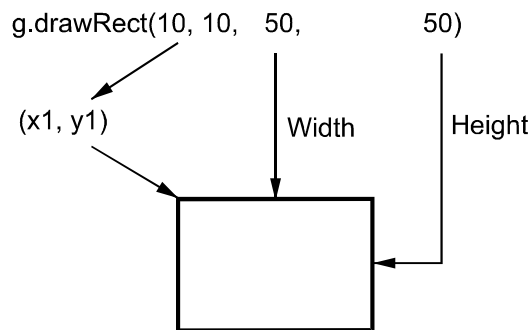


Fig. 2.17.3 Rectangle

The rectangle can be filled up with following methods,

```
void fillRect(int top,int left,int width,int height)
```

```
void fillRoundRect(int top,int left,int width,int height,int xdimeter,int ydimeter)
```

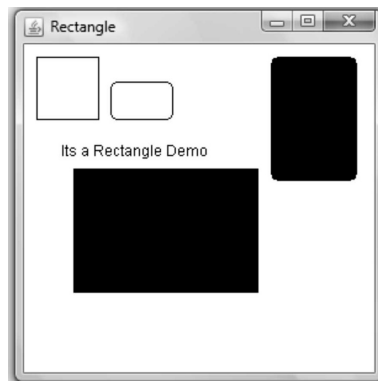
Java Program[RectangleDemo.java]

```
import java.awt.*;  
class RectangleDemo extends Canvas  
{  
    public RectangleDemo()  
    {
```



```
setSize(200,200);
setBackground(Color.white);
}
public static void main(String[] args)
{
    RectangleDemo obj=new RectangleDemo();
    Frame fr=new Frame("Rectangle");
    fr.setSize(300,300);
    fr.add(obj);
    fr.setVisible(true);
}
public void paint(Graphics g)
{
    g.drawRect(10,10,50,50);
    g.drawRoundRect(70,30,50,30,10,10);
    g.fillRect(40,100,150,100);
    g.fillRoundRect(200,10,70,100,10,10);
    g.drawString("Its a Rectangle Demo",30,90);
}
}
```

Output



2.17.3 Oval

To draw circle and ellipse we can use the function **drawOval()** method. The syntax of this method is ,

```
void drawOval(int top, int left, int width, int height)
```

To fill this oval we use **fillOval()** method. The syntax of this method is,

```
void fillOval(int top, int left, int width, int height)
```

We can specify the color for filling up the rectangle using **setColor** method. For example

```
g.drawOval(10,10,200,100);  
g.setColor(Color.blue);  
g.fillOval(30,50,200,100);
```

The *top* and *left* values specify the upper left corner and *width* and *height* is for specifying the width and heights respectively.

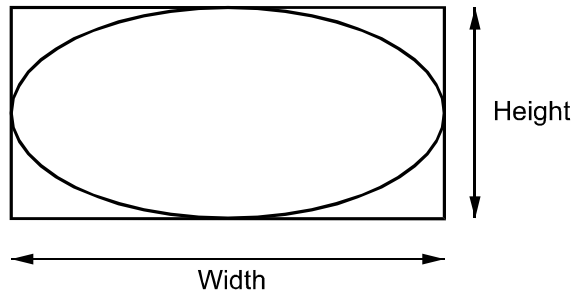
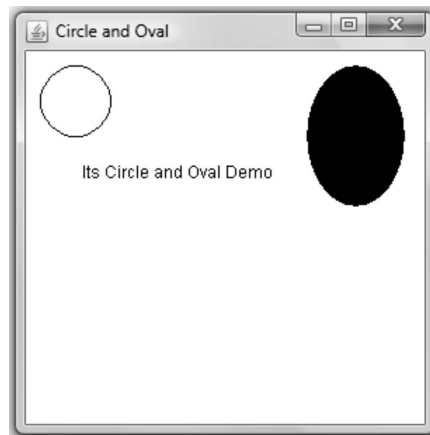


Fig. 2.17.4 Ellipse

Let us understand the functioning of these methods with the help of some example

Java Program[OvalDemo.java]

```
import java.awt.*;  
class OvalDemo extends Canvas  
{  
    public OvalDemo()  
    {  
        setSize(200,200);  
        setBackground(Color.white);  
    }  
    public static void main(String[] args)  
    {  
        OvalDemo obj=new OvalDemo();  
        Frame fr=new Frame("Circle and Oval");  
        fr.setSize(300,300);  
        fr.add(obj);  
        fr.setVisible(true);  
    }  
    public void paint(Graphics g)  
    {  
        g.drawOval(10,10,50,50);  
        g.fillOval(200,10,70,100);  
        g.drawString("Its Circle and Oval Demo",40,90);  
    }  
}
```

Output**2.17.4 Arc**

To draw an arc the **drawArc()** and to fill an arc **fillArc()** are the functions. The syntax of these methods is as follows -

```
void drawArc(int top,int left,int width,int height,int angle1,int angle2)
```

```
void fillArc(int top,int left,int width,int height,int angle1,int angle2)
```

The *angle1* represents the starting angle and the *angle2* represents the angular distance.

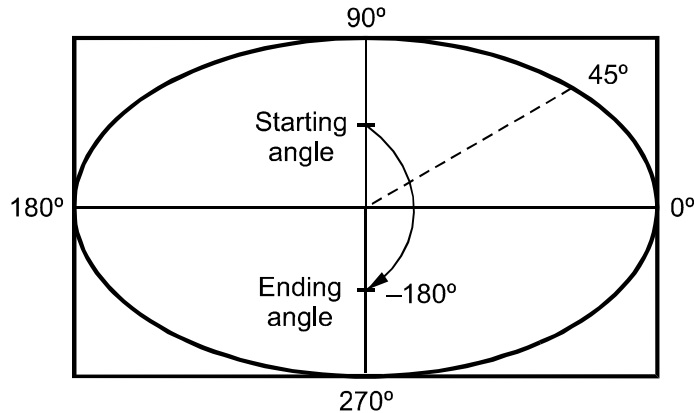


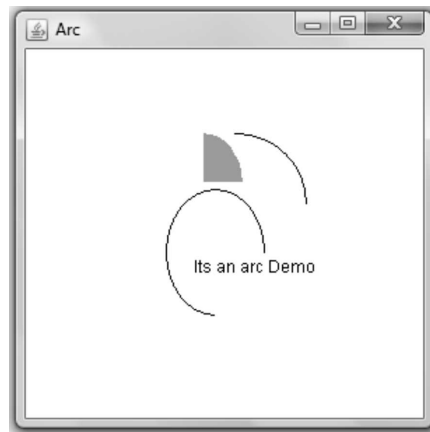
Fig. 2.17.5

Java Program[arcDemo.java]

```
import java.awt.*;
class arcDemo extends Canvas
{
    public arcDemo()
    {
        setSize(200,200);
    }
}
```

```
setBackground(Color.white);
}
public static void main(String[] args)
{
    arcDemo obj=new arcDemo();
    Frame fr=new Frame("Arc");
    fr.setSize(300,300);
    fr.add(obj);
    fr.setVisible(true);
}
public void paint(Graphics g)
{
    g.drawArc(100,60,100,100,0,90);
    g.setColor(Color.green);//fills the arc with green
    g.fillArc(100,60,55,70,0,90);
    g.setColor(Color.black);
    g.drawArc(100,100,70,90,0,270);
    g.drawString("Its an arc Demo",120,160);
}
}
```

Output



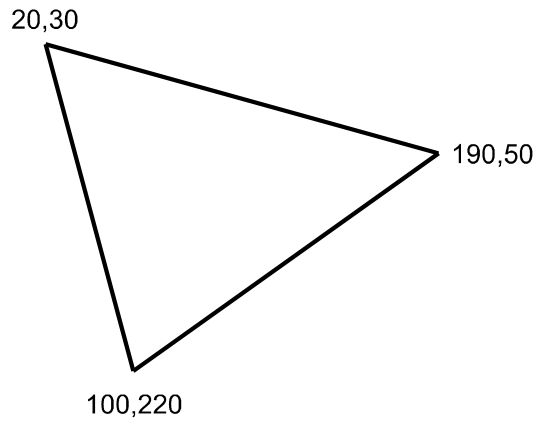
2.17.5 Polygons

In order to draw polygons following function is used

```
Polygon(int[] xpoints,int[] ypoints,int npoints)
```

The *xpoints* represent the array of *x* co-ordinates. The *ypoints* represent the array of *y* co-ordinates. And the *npoints* represents the number of points.

For filling up the polygon the **fillPolygon** method is used.

**Java Program[Polyg.java]**

```
import java.awt.*;
class Polyg extends Canvas
{
    public Polyg()
    {
        setSize(200,200);
        setBackground(Color.white);
    }
    public static void main(String[] args)
    {
        Polyg obj=new Polyg();
        Frame fr=new Frame("Polygon");
        fr.setSize(300,300);
        fr.add(obj);
        fr.setVisible(true);
    }
    public void paint(Graphics g)
    {
        int xpt[]={50,20,20,20,130};
        int ypt[]={80,30,200,200,30};
        int num=5;
        g.drawPolygon(xpt,ypt,num);
        g.setColor(Color.magenta);
        g.fillPolygon(xpt,ypt,num);
        g.setColor(Color.black);
        g.drawString("Its a polygon Demo",100,100);
    }
}
```

Output**Review Questions**

1. List the methods available in the draw shapes.
2. Write the short note on - Graphics programming.
3. How will you draw the following graphics in a window ?
i) Arcs ii) Ellipses and circles in Java.
4. With an example describe in detail about how to work with 2D Shapes in Java.

2.18 Layout Manager**Definition :**

- A **Layout manager** is an interface which automatically arranges the controls on the screen.
- Thus using layout manager the **symmetric** and **systematic arrangement** of the **controls** is possible. In this section we will discuss following layout managers.

2.18.1 FlowLayout

- FlowLayout manager is the simplest Layout manager.
- Using this Layout manager components are arranged from top left corner lying down from left to right and top to bottom.
- Between each component there is some space left.
- The **syntax** of FlowLayout manager is as given below -

```
FlowLayout(int alignment)
```

Where *alignment* denotes the alignment of the components on the applet windows. The alignment can be denoted as :

FlowLayout.LEFT
FlowLayout.RIGHT
FlowLayout.CENTER

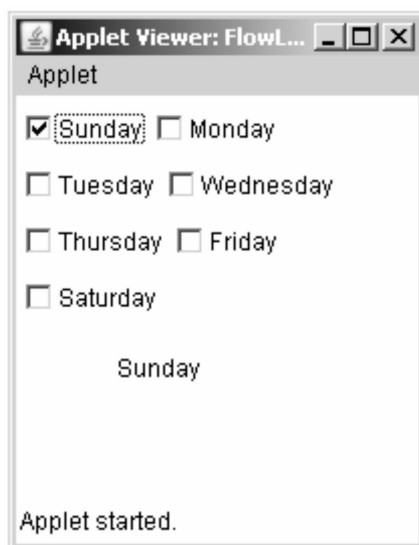
- Here is a Java program which makes use of seven checkboxes which are aligned on the applet window using FlowLayout manager.

Java Program[ItemListener.java]

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="FlowLDemo" width=200 height=200>
</applet>
*/
public class FlowLDemo extends Applet
implements ItemListener
{
    String msg=" ";
    Checkbox box1=new Checkbox("Sunday");
    Checkbox box2=new Checkbox("Monday");
    Checkbox box3=new Checkbox("Tuesday");
    Checkbox box4=new Checkbox("Wednesday");
    Checkbox box5=new Checkbox("Thursday");
    Checkbox box6=new Checkbox("Friday");
    Checkbox box7=new Checkbox("Saturday");
    public void init()
    {
        //creating FlowLayout manager
        setLayout(new FlowLayout(FlowLayout.LEFT));
        //adding the components with Left alignment
        add(box1);
        add(box2);
        add(box3);
        add(box4);
        add(box5);
        add(box6);
        add(box7);
        //registering the checkboxes to EventListener
        box1.addItemListener(this);
        box2.addItemListener(this);
        box3.addItemListener(this);
        box4.addItemListener(this);
        box5.addItemListener(this);
```

```
box6.addItemListener(this);
box7.addItemListener(this);
}
public void paint(Graphics g)
{
    //if box1 checkbox is clicked
    if(box1.getState())
        msg="Sunday ";//then print the corresponding day
    if(box2.getState())
        msg="Monday ";
    if(box3.getState())
        msg="Tuesday ";
    if(box4.getState())
        msg="Wednesday ";
    if(box5.getState())
        msg="Thursday ";
    if(box6.getState())
        msg="Friday ";
    if(box7.getState())
        msg="Saturday ";
    g.drawString(msg,50,140);
}
public void itemStateChanged(ItemEvent e)
{
    repaint();
}
}
```

Output



Program Explanation :

In above program, since we are using Check box control the **ItemListener** interface is used. And for registering these controls to receive the events we have written following lines in the **init** method.

```
box1.addItemListener(this);
box2.addItemListener(this);
box3.addItemListener(this);
box4.addItemListener(this);
box5.addItemListener(this);
box6.addItemListener(this);
box7.addItemListener(this);
```

We have written a method **public void itemStateChanged (ItemEvent e)** because **ItemListener** interface is used.

2.18.2 BorderLayout

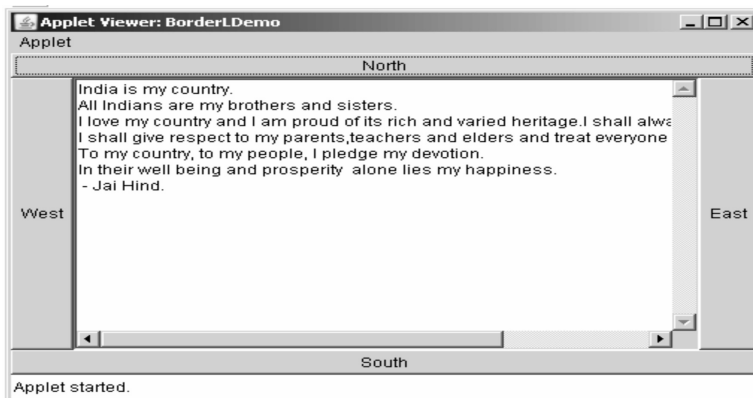
- In BorderLayout there are **four components** at the four sides and one component occupying large area at the centre.
- The central area is called CENTER and the components forming four sides are called LEFT,RIGHT,TOP and BOTTOM.
- Following program consists of one big message stored in variable **msg** which is to be displayed at the centre. And in the **init** method it shows that four sides are occupied by the **Buttons**.

Java Program[BorderLDemo.java]

```
import java.applet.*;
import java.awt.*;
import java.util.*;
/*
<applet code="BorderLDemo" width=500 height=300>
</applet>
*/
public class BorderLDemo extends Applet
{
String msg="India is my country.\n"+"All Indians are my
    brothers and sisters.\n"+
    "I love my country and I am proud of its rich
    and varied heritage."+
    "I shall always strive to be worthy of it.\n"+
    "I shall give respect to my parents,teachers and
    elders and treat everyone with courtesy.\n"+
```

```
"To my country, to my people, I pledge my
devotion.\n"+
"In their well being and prosperity alone lies my
happiness.\n"+
" - Jai Hind.";
public void init()
{
    setLayout(new BorderLayout());
    add(new Button("North"),BorderLayout.NORTH);
    add(new Button("South"),BorderLayout.SOUTH);
    add(new Button("East"),BorderLayout.EAST);
    add(new Button("West"),BorderLayout.WEST);
    add(new TextArea(msg),BorderLayout.CENTER);
}
}
```

Output



Program Explanation :

In the above program, we have created object for BorderLayout manager using

```
setLayout(new BorderLayout());
```

Then using

```
BORDER.NORTH
BORDER.SOUTH
BORDER.EAST
BORDER.WEST
```

- The four sides are set with the help of **Button** control. The central large area is formed using **TextArea** control which is called as **BORDER.CENTER**.
- The concept of BorderLayout can then be clearly understood with the help of above given output.

- In this Layout manager, we can add one more method called **getInsets()**. This method allows us to leave some space between underlying window on the applet and Layout manager.
- We have used this method in the following program. The syntax of **Insets** method is

```
Insets(int top,int left,int bottom,int right)
```

The *top*,*left*,*bottom* and *right* parameters specify the amount of space to be left.

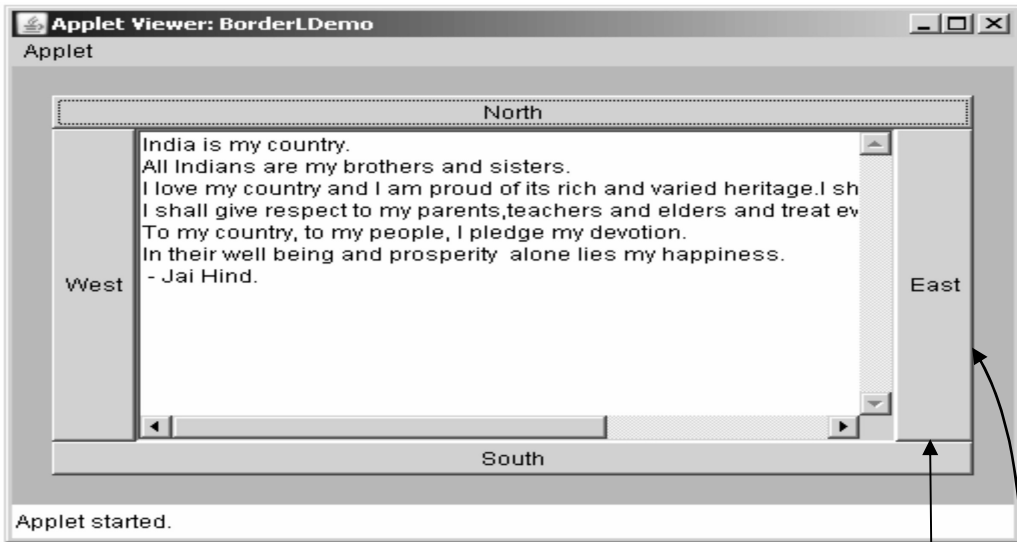
Java Program

```
import java.applet.*;
import java.awt.*;
import java.util.*;
/*
<applet code="BorderLDemo" width=500 height=300>
</applet>
*/
public class BorderLDemo extends Applet
{
    String msg="India is my country.\n"+"All Indians are my
        brothers and sisters.\n"+
        "I love my country and I am proud of its rich
        and varied heritage."+
        "I shall always strive to be worthy of it.\n"+
        "I shall give respect to my parents,teachers
        and elders and treat everyone with
        courtesy.\n"+
        "To my country, to my people, I pledge my
        devotion.\n"+
        "In their well being and prosperity alone lies
        my happiness.\n"+
        "- Jai Hind.";
    public void init()
    {
        setBackground(Color.green);
        setLayout(new BorderLayout());
        add(new Button("North"),BorderLayout.NORTH);
        add(new Button("South"),BorderLayout.SOUTH);
        add(new Button("East"),BorderLayout.EAST);
        add(new Button("West"),BorderLayout.WEST);
        add(new TextArea(msg),BorderLayout.CENTER);
    }
    public Insets getInsets()
    {
        return new Insets(20,20,20,20);
    }
}
```

```

}
}

```

Output

This Space is left in between

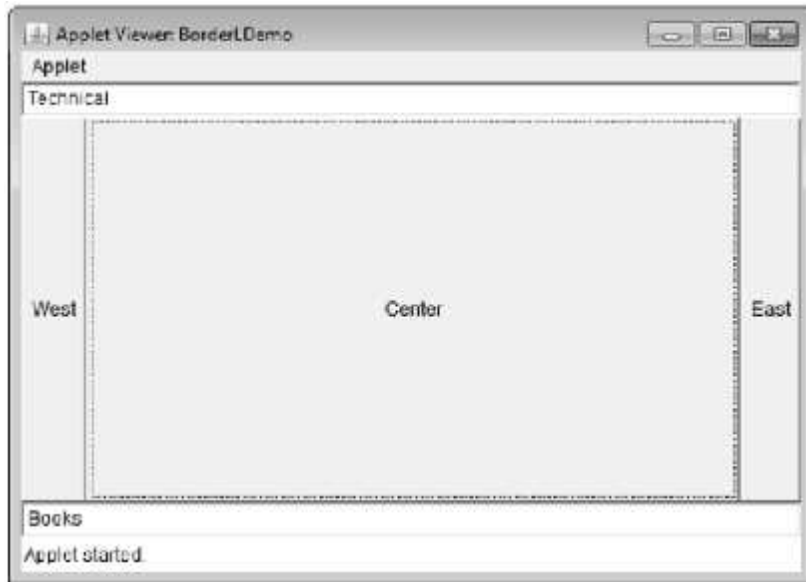
Example 2.18.1 Write a Java program which create border layout and adds two text boxes to it.

Solution :**BorderLDemo.java**

```

import java.applet.*;
import java.awt.*;
import java.util.*;
/*
<applet code="BorderLDemo" width=500 height=300>
</applet>
*/
public class BorderLDemo extends Applet
{
    public void init()
    {
        setLayout(new BorderLayout());
        add(new Button("Center"),BorderLayout.CENTER);
        add(new Button("East"),BorderLayout.EAST);
        add(new Button("West"),BorderLayout.WEST);
        add(new TextField("Technical"),BorderLayout.NORTH);
        add(new TextField("Books"),BorderLayout.SOUTH);
    }
}

```

Output**2.18.3 GridLayout**

- GridLayout is a Layout manager used to arrange the components in a grid. The syntax of GridLayout manager is

```
GridLayout(int n,int m)
```

Where n represents total number of rows and m represents total number of columns.

- In the following program we have arranged **Buttons** in a grid form.

Java Program[GridLDemo.java]

```
import java.awt.*;
import java.applet.*;
/*
<applet code="GridLDemo" width=400 height=400>
</applet>
*/
public class GridLDemo extends Applet
{
    int n=4,m=3;
    public void init()
    {
        setLayout(new GridLayout(n,n));
        for(int i=0;i<n;i++)
```

```
{
for(int j=0;j<m;j++)
{
switch(i)
{
case 0:if(j==0)          //Button[0,0]
        add(new Button("Red"));
        else if(j==1)    //Button[0,1]
        add(new Button("Green"));
        else if(j==2)      //Button[0,2]
        add(new Button("Blue"));
        break;
case 1:if(j==0)          //Button[1,0]
        add(new Button("Orange"));
        else if(j==1)    //Button[1,1]
        add(new Button("Pink"));
        else if(j==2)      //Button[1,2]
        add(new Button("Magenta"));
        break;
case 2:if(j==0)          //Button[2,0]
        add(new Button("Cyan"));
        else if(j==1)    //Button[2,1]
        add(new Button("Gray"));
        else if(j==2)      //Button[2,2]
        add(new Button("Yellow"));
        break;
case 3:if(j==0)          //Button[3,0]
        add(new Button("Black"));
        else if(j==1)    //Button[3,1]
        add(new Button("White"));
        else if(j==2)      //Button[3,2]
        add(new Button("LightGray"));
        break;
} //end of switch
} //end of inner for
} //end of outer for
} // end of function init
} //end for class
```

Output**2.18.4 CardLayout**

- Sometimes we want to perform various sets of graphical controls at a time then CardLayout is used.
- Thus CardLayout manager allows us to have more than one layouts on the applet.
- The CardLayout is conceptually thought as a collection of cards lying on a **panel**.
- We have to follow following steps -

Step 1 : We have to create two objects

1. Panel object
2. CardLayout object

Step 2 : Then we have to add the cards on the panel using **add()** method. For example -

```
panel_obj.setLayout(layout_obj);
```

where *panel_obj* is an object of panel and *layout_obj* is an object of CardLayout

Step 3 : Finally we have to add the object of panel to main applet. For example -

```
add(panel_obj);
```

These all stages seem to be complicated. Hence let us understand following program which implements CardLayout.

Java Program[cardDemo.java]

```
//This program demonstrates cardLayout
//The dynamic selection of fruit/flower/colour can be made
//using cardlayout component
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="cardDemo" width=300 height=100>
</applet>
*/
public class cardDemo extends Applet implements
ActionListener,MouseListener
{
    Checkbox mango,apple,rose,lotus,Red,Green;
    Panel panel_obj;
    CardLayout layout_obj;
    Button fruit,flower,colour;
    public void init()
    {
        fruit=new Button("Fruit");
        flower=new Button("Flower");
        colour=new Button("Colour");
        //adding the button controls
        add(fruit);
        add(flower);
        add(colour);
        //getting object of Cardlayout
        layout_obj=new CardLayout();
        //getting object of Panel
        panel_obj=new Panel();
        panel_obj.setLayout(layout_obj);
        //adding checkbox controls for fruits
        mango=new Checkbox("Mango");
        apple=new Checkbox("Apple");
        //adding checkbox controls for flowers
        rose=new Checkbox("Rose");
        lotus=new Checkbox("Lotus");
        //adding checkbox controls for colors
```



```
Red=new Checkbox("Red");
Green=new Checkbox("Green");

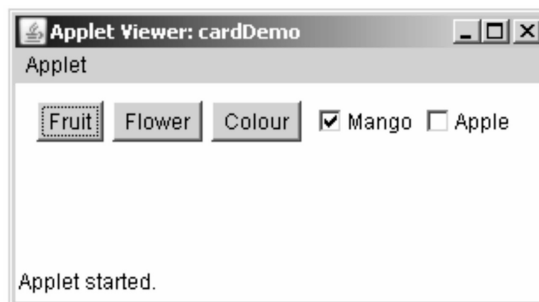
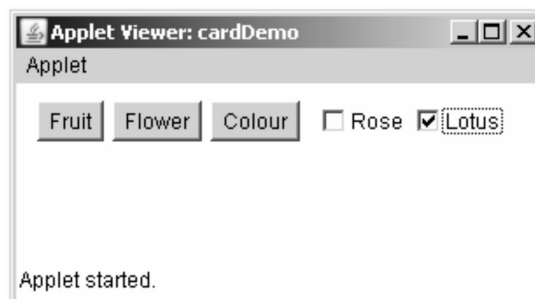
Panel fruit_pan=new Panel();
fruit_pan.add(mango);
fruit_pan.add(apple);

Panel flower_pan=new Panel();
flower_pan.add(rose);
flower_pan.add(lotus);

Panel colour_pan=new Panel();
colour_pan.add(Red);
colour_pan.add(Green);

panel_obj.add(fruit_pan,"Fruit");
panel_obj.add(flower_pan,"Flower");
panel_obj.add(colour_pan,"Colour");
add(panel_obj);
//register the components to event listener
fruit.addActionListener(this);
flower.addActionListener(this);
colour.addActionListener(this);
addMouseListener(this);
}
//following empty methods are necessary for mouse events
public void mousePressed(MouseEvent m)
{
    layout_obj.next(panel_obj);
}
public void mouseClicked(MouseEvent m)
{
}
public void mouseEntered(MouseEvent m)
{
}
public void mouseExited(MouseEvent m)
{
}
public void mouseReleased(MouseEvent m)
{
}
public void actionPerformed(ActionEvent e)
{
}
```

```
if(e.getSource()==fruit)
{
    layout_obj.show(panel_obj,"Fruit");
}
else if(e.getSource()==flower)
{
    layout_obj.show(panel_obj,"Flower");
}
else if(e.getSource()==colour)
{
    layout_obj.show(panel_obj,"Colour");
}
}
} //end for actionPerformed method
} //end of class
```

Output(Run 1)**Output(Run 2)****Output (Run 3)**

Program Explanation :

- It is clear from the output that we can have a combination of various components lying on the same applet and can be invoked as per need. That mean if we click on *Fruit* button then we should get two checkboxes namely : *Mango* and *Apple*.
- If we click on *Flower* button then we should get two checkboxes namely : *Rose* and *Lotus*. Similarly, if we click on *Colour* button we should get two checkboxes namely : *Red* and *Green*.
- In above program, we have used to event listener interfaces : **ActionListener** and **MouseListener**.
- We have created **panel object** and **Layout object**.
- In the **init** method we have first created and added the **Button** controls. Then **CardLayout** is placed on the panel.
- The panel is then set for the applet window.
- Various components such as **checkboxes** and **Buttons** are added to the panel.
- In order to understand mouse events some necessary empty methods are written.
- In the **actionPerformed()** method, on the click of **Fruit** button, two corresponding check boxes are shown. Same is true for **Flower** and **Colour** buttons.

2.18.5 GridBagLayout

- The GridBagLayout is the most flexible and complex layout manager.
- The GridBagLayout manager places the components in rows and columns allowing the components to occupy multiple rows and columns. This is called **display area**.
- GridBagLayout performs three functions using values from the **GridBagConstraints** parameter in the **add()** method.
 1. Grid position, width and height describe the display area using **gridx**, **gridy**, **Gridwidth** and **gridheight** values.
 2. Position within the display area using fill, **ipadx** and **ipady**.
 3. Identifying rows and columns which receive extra space on expansion using **weightx**, and **weighty**.
- The layout can be set as follows

```
Container pane=frame.getContentPane();  
pane.setLayout(new GridBagLayout());
```

- The component can be added as

```
pane.add(component,constraintObject);//pane is a container pane
```

- The following values are then set -
 - **gridx and gridy** : These values denote the integer column and row value of the component.
 - **gridwidth and gridheight** : They denote the number of columns and rows the component occupies.
 - **weightx and weighty** : They denote the extra space occupied by the component horizontally or vertically when the output window is resized.
 - **fill** : The fill value denotes how the component should expand within the display area.

Typical values are -

GridBagConstraints.NONE // Can not expand (Default)

GridBagConstraints.VERTICAL // Expand vertically

GridBagConstraints.HORIZONTAL // Expand horizontally

GridBagConstraints.BOTH // Expand vertically and horizontally

- **ipadx and ipady** : These values denote increase and decrease in horizontal or vertical preferred size of the component. Default value is 0.

Example 2.18.2 Write a simple Java program that illustrates the use of GridBagLayout.

Solution :

```
import java.awt.*;
import java.applet.*;
/*
<applet code="GridBagLayoutDemo" width=400 height=400>
</applet>
*/
public class GridBagLayoutDemo extends Applet
{
    public void init()
    {
        Button B;
        setLayout(new GridBagLayout());
        GridBagConstraints gBC = new GridBagConstraints();
        gBC.fill = GridBagConstraints.HORIZONTAL; //placing the components horizontally
        B = new Button("Button 1");//first component
        gBC.weightx = 0.5;
        gBC.gridx = 0;
        gBC.gridy = 0;
        add(B, gBC);

        B = new Button("Button 2");//second component
        gBC.gridx = 2;
        gBC.gridy = 0;
        add(B, gBC);
    }
}
```

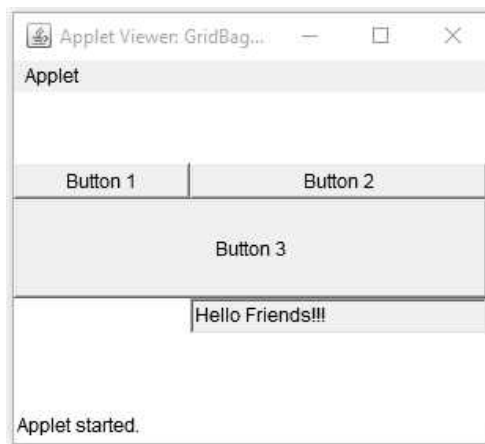
```
B = new Button("Button 3"); //third component
gBC.ipady = 40; //This component is broad
gBC.weightx = 0.0;
gBC.gridwidth = 3;
gBC.gridx = 0;
gBC.gridy = 1;
add(B, gBC);

TextField T = new TextField("Hello Friends!!!");//forth component
gBC.ipady = 0;
gBC.weightx = 0.0;
gBC.gridx = 1;
gBC.gridwidth = 2;
gBC.gridy = 2;
T.setEditable(false); //text field is not editable
add(T, gBC);
}
```

Output



Before Expanding



After Expanding

Review Questions

1. What is the function of layout manager ? Describe in detail about the different layout in Java GUL.
2. What is layout management ? State the various types of layout supported by JAVA. Which layout is default one ? Discuss the components of swing.

2.19 Multiple Choice Questions

Q.1 Give the Abbreviation of AWT :

- | | |
|---|---|
| ☐ a Applet Windowing Toolkit | ☐ b Abstract Windowing Toolkit |
| ☐ c Absolute Windowing Toolkit | ☐ d None of these |

Q.2 In which places can put the event handling code _____.

- | | |
|--|---|
| ☐ a same class | ☐ b other class |
| ☐ c anonymous class | ☐ d all of the above |

Q.3 In Java an event is an _____ which specifies the change of state in the source.

- | | |
|----------------------------------|-----------------------------------|
| ☐ a class | ☐ b object |
| ☐ c int | ☐ d string |

Q.4 The Following steps are required to perform 1) Implement the listener interface and overrides its methods 2) Register the component with the listener

- | | |
|---|--|
| ☐ a Exception Handling | ☐ b String Handling |
| ☐ c Event Handling | ☐ d None of the above |

Q.5 Various event classes and listener interfaces for event handling are present in following package :

- | | |
|---|--|
| ☐ a java.awt | ☐ b java.awt.graphics |
| ☐ c java.awt.event | ☐ d None of these |

Q.6 The ActionListener interface is used for handling action events by _____.

- | | |
|--------------------------------------|---|
| ☐ a JButton | ☐ b JCheckbox |
| ☐ c JMenuItem | ☐ d All of these |

Q.7 Which is the container that doesn't contain title bar and MenuBars. It can have other components like button, textfield etc ?

- | | |
|-----------------------------------|--------------------------------------|
| ☐ a Window | ☐ b Frame |
| ☐ c Panel | ☐ d Container |

Q.8 Clicking a mouse button will always generate which event ?

- | | |
|--|--|
| ☐ a MouseEvent | ☐ b ActionEvent |
| ☐ c MouseClickEvent | ☐ d MouseEvent |

Q.9 By which method we can set or change the text in a Label in AWT ?

- | | |
|--------------------------------------|---|
| ☐ a setText() | ☐ b getText() |
| ☐ c addText() | ☐ d all of these |

Q.10 Which layout manager places components in one of five regions : north, south, east, west, and center ?

☐ a) BorderLayout

☐ b) GridLayout

☐ c) FlowLayout

☐ d) GridLayout

Q.11 Arranges the components horizontally :

☐ a) BorderLayout

☐ b) CardLayout

☐ c) GridLayout

☐ d) FlowLayout

Q.12 The most commonly used layout managers are _____.

☐ a) FlowLayout

☐ b) BorderLayout

☐ c) GridLayout

☐ d) All of these

Q.13 Default layout manager for subclasses of window is _____.

☐ a) cardlayout

☐ b) GridBagLayout

☐ c) Frame

☐ d) BorderLayout

Answer Keys for Multiple Choice Questions :

Q.1	b	Q.2	d	Q.3	b	Q.4	c
Q.5	c	Q.6	d	Q.7	c	Q.8	d
Q.9	a	Q.10	c	Q.11	d	Q.12	d
Q.13	d						



Notes

[illegible]